

DATA ANALYTICS WITH COGNOS - GROUP 5

PROJECT: CUSTOMER CHURN PREDICTION

PHASE 4: DEVELOPMENT PART 2

TEAM MEMBER

963321104011: ASWINI A J

PHASE 4 – DOCUMENT SUBMISSION

**TOPIC : IN THIS SECTION CONTINUE BUILDING THE PROJECT
BY PERFORMING DIFFERENT ACTIVITIES LIKE FEATURE
ENGINEERING, MODEL TRAINING, EVALUATION etc.**



CUSTOMER CHURN PREDICTION

INTRODUCTION

In today's highly competitive business landscape, retaining customers is of paramount importance. Customer churn, the rate at which customers discontinue their relationship with a company, can significantly impact a company's profitability and long-term success. To address this challenge, this project focuses on customer churn prediction. By leveraging data analytics and machine learning techniques, we aim to develop a predictive model that can identify customers at risk of churning. This model will enable businesses to take proactive measures to retain valuable customers, ultimately increasing customer loyalty and sustaining growth. In this project, we will explore the data, develop predictive models, and provide actionable insights to reduce churn and enhance customer satisfaction.

As we embark on the development phase of our customer churn prediction project, we move closer to realizing the goal of building a robust and data-driven solution to address one of the most pressing challenges in the business world. The development phase represents a critical juncture in our project journey. During this phase, we will leverage cutting-edge data analytics and machine learning techniques to transform raw data into actionable insights. Our mission is to create a predictive model capable of identifying customers on the brink of churning, empowering businesses to implement timely interventions. This phase involves data preprocessing, feature engineering, model selection, and rigorous testing to ensure the accuracy and effectiveness of our churn prediction system. As we delve deeper into the development process, we are one step closer to delivering a tool that will not only reduce churn rates but also foster lasting customer relationships, driving business success in an increasingly competitive market.

DATASET

Telco customer churn:

The dataset contains 7043 rows and 21 features. Each row represents a customer, each column contains the customer's attributes described in the column Metadata

Dataset Link: <https://www.kaggle.com/datasets/blastchar/telco-customer-churn>

customer	gender	SeniorCiti	Partner	Depender	tenure	PhoneSer	Multiple	Li	InternetS	OnlineSec	OnlineBac	DevicePrc	TechSupp	Streaming	Streaming	Contract	Paperless	Paymenth	MonthlyC	TotalChar	Churr
7590-VHV	Female	0	Yes	No	1	No	No phone	DSL	No	Yes	No	No	No	No	No	Month-to	Yes	Electronic	29.85	29.85	No
5575-GNV	Male	0	No	No	34	Yes	No	DSL	Yes	No	Yes	No	No	No	No	One year	No	Mailed ch	56.95	1889.5	No
3668-QPY	Male	0	No	No	2	Yes	No	DSL	Yes	Yes	No	No	No	No	No	Month-to	Yes	Mailed ch	53.85	108.15	Yes
7795-CFO	Male	0	No	No	45	No	No phone	DSL	Yes	No	Yes	Yes	No	No	No	One year	No	Bank trans	42.3	1840.75	No
9237-HQT	Female	0	No	No	2	Yes	No	Fiber opti	No	No	No	No	No	No	No	Month-to	Yes	Electronic	70.7	151.65	Yes
9305-CD5	Female	0	No	No	8	Yes	Yes	Fiber opti	No	No	Yes	No	Yes	Yes	Yes	Month-to	Yes	Electronic	99.65	820.5	Yes
1452-KIO	Male	0	No	Yes	22	Yes	Yes	Fiber opti	No	Yes	No	No	Yes	No	No	Month-to	Yes	Credit car	89.1	1949.4	No
6713-OKO	Female	0	No	No	10	No	No phone	DSL	Yes	No	No	No	No	No	No	Month-to	No	Mailed ch	29.75	301.9	No
7892-POO	Female	0	Yes	No	28	Yes	Yes	Fiber opti	No	No	Yes	Yes	Yes	Yes	Yes	Month-to	Yes	Electronic	104.8	3046.05	Yes
6388-TAB	Male	0	No	Yes	62	Yes	No	DSL	Yes	Yes	No	No	No	No	No	One year	No	Bank trans	56.15	3487.95	No
9763-GR5	Male	0	Yes	Yes	13	Yes	No	DSL	Yes	No	No	No	No	No	No	Month-to	Yes	Mailed ch	49.95	587.45	No
7469-LKB	Male	0	No	No	16	Yes	No	No	No intern	No intern	No intern	No intern	No intern	No intern	No intern	Two year	No	Credit car	18.95	326.8	No
8091-TTV	Male	0	Yes	No	58	Yes	Yes	Fiber opti	No	No	Yes	No	Yes	Yes	Yes	One year	No	Credit car	100.35	5681.1	No
0280-XJG	Male	0	No	No	49	Yes	Yes	Fiber opti	No	Yes	Yes	No	Yes	Yes	Yes	Month-to	Yes	Bank trans	103.7	5036.3	Yes
5129-JLP	Male	0	No	No	25	Yes	No	Fiber opti	Yes	No	Yes	Yes	Yes	Yes	Yes	Month-to	Yes	Electronic	105.5	2686.05	No
3655-SNQ	Female	0	Yes	Yes	69	Yes	Yes	Fiber opti	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Two year	No	Credit car	113.25	7895.15	No
8191-XWS	Female	0	No	No	52	Yes	No	No	No intern	No intern	No intern	No intern	No intern	No intern	No intern	One year	No	Mailed ch	20.65	1022.95	No
9959-WOF	Male	0	No	Yes	71	Yes	Yes	Fiber opti	Yes	No	Yes	No	Yes	Yes	Yes	Two year	No	Bank trans	106.7	7382.25	No
4190-MFL	Female	0	Yes	Yes	10	Yes	No	DSL	No	No	Yes	Yes	No	No	No	Month-to	No	Credit car	55.2	528.35	Yes
4183-MYF	Female	0	No	No	21	Yes	No	Fiber opti	No	Yes	Yes	No	No	Yes	Yes	Month-to	Yes	Electronic	90.05	1862.9	No

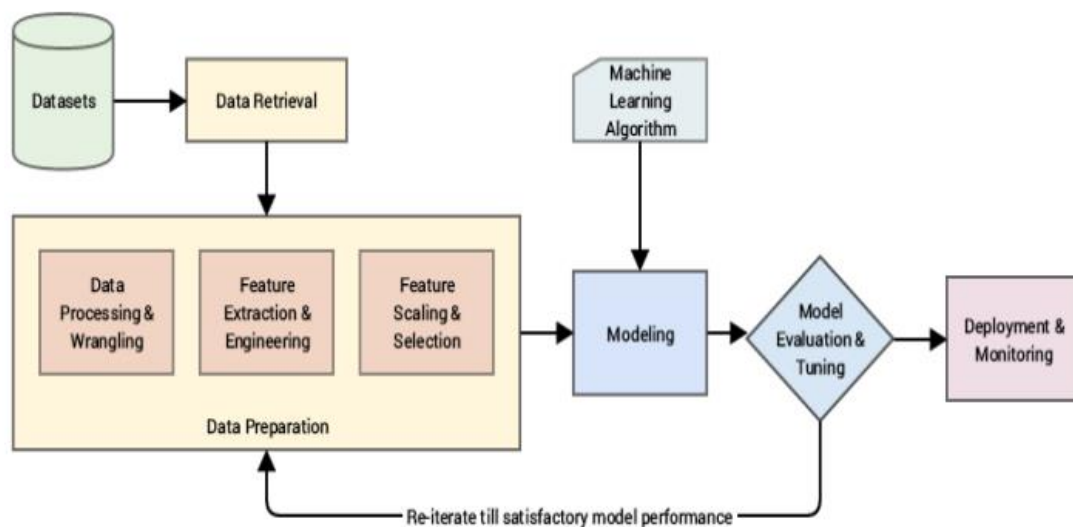
OVERVIEW OF THE PROCESS

The development phase of a customer churn prediction project typically involves several key steps:

- 1. Data Collection:** Gather relevant data, such as customer demographics, transaction history, and customer interactions.
- 2. Feature Engineering:** Create new features or transform existing ones to make the data more suitable for predictive modeling.
- 3. Model Selection:** Choose appropriate machine learning or statistical models for churn prediction, such as logistic regression, decision trees, random forests, or neural networks.
- 4. Model Training:** Train the selected models on the training data using appropriate algorithms.
- 5. Model Evaluation:** Assess model performance on the validation set using metrics like accuracy, precision, recall, and F1-score. Adjust models as needed.
- 6. Deployment:** Implement the model into the production environment, integrating it with the organization's systems.

FEATURE ENGINEERING

Feature engineering refers to the process of using domain knowledge to select and transform the most relevant variables from raw data when creating a predictive model using machine learning or statistical modeling. The goal of feature engineering and selection is to improve the performance of machine learning (ML) algorithms.



It is the process of transforming raw data into features that are suitable for machine learning models. In other words, it is the process of selecting, extracting, and transforming the most relevant features from the available data to build more accurate and efficient machine learning models.

Program

```
# Remove Gender
```

```
Features = ['gender']
```

```
Df_processed.drop(features, axis=1, inplace=True)
```

```
Logit = Model(LogisticRegression, df_processed, metrics)
```

```
Print_result(logit.grid_search('roc_auc', logit_grid), "cross-validation")
```

MODEL TRAINING

Model training is the process of teaching a machine learning model to predict customer churn. It involves in the field of marketing, scoring is a technique that enables one to assign a score to a customer or a prospect. The corresponding score generally reflects the probability that a customer or prospect will answer a marketing solicitation. Predictive scoring can be used to identify customers at high risk of attribution.

Choose A Machine Learning Algorithm

There are a number of different machine learning algorithms that can be used for customer churn prediction such as Logistic Regression, Bayes Algorithm, Decision Trees, Support Vector Machines, Linear Discriminant Analysis

Machine Learning Models

Logistic Regression:

In:

```
import warnings

warnings.filterwarnings('ignore')

from sklearn.feature_selection import RFE

from sklearn.linear_model import LogisticRegression

X = data[all_columns] # Features

y = data['Churn'] # Target variable

model = LogisticRegression()

rfe = RFE(model, 8)

fit = rfe.fit(X, y)

print("Num Features: %s" % (fit.n_features_))

print("Selected Features: %s" % (fit.support_))

print("Feature Ranking: %s" % (fit.ranking_))
```

Out:

Num Features: 8

Selected Features: [False True False True False True False False True True False True
False False True True False False False]

Feature Ranking: [3 1 10 1 4 1 2 7 1 1 5 1 11 9 1 1 6 8 12]

In:

```
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn import metrics

final_features_rfe = []

for status, var in zip(selected_features_rfe, all_columns):

    if status == True:

        final_features_rfe.append(var)

final_features_rfe
```

Out:

```
['SeniorCitizen',
 'Dependents',
 'PhoneService',
 'OnlineSecurity',
 'OnlineBackup',
 'TechSupport',
 'Contract',
 'PaperlessBilling']
```

In:

```
X_rfe_lr = data[final_features_rfe]

y = data['Churn']

X_train_rfe_lr,X_test_rfe_lr,y_train_rfe_lr,y_test_rfe_lr=train_test_split(X_rfe_lr,y,test_size=
0.25,random_state=0)

lr_model = LogisticRegression()

# fit the model with data

lr_model.fit(X_train_rfe_lr,y_train_rfe_lr)

y_pred_rfe_lr=lr_model.predict(X_test_rfe_lr)

acc_rfe_lr = metrics.accuracy_score(y_test_rfe_lr, y_pred_rfe_lr)

print("Accuracy: ",acc_rfe_lr)
```

Out:

Accuracy: 0.7677455990914254

In:

```
X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.25,random_state=0)

# instantiate the model (using the default parameters)

lr_model_single = LogisticRegression()

# fit the model with data

lr_model_single.fit(X_train,y_train)

y_pred=lr_model_single.predict(X_test)

lr_acc = metrics.accuracy_score(y_test, y_pred)

print("Accuracy: ",lr_acc)
```

Out:

Accuracy: 0.7938671209540034

Bayes Algorithm:

In:

```
from sklearn.naive_bayes import GaussianNB

from sklearn.metrics import accuracy_score

model = GaussianNB()

# fit the model with the training data

model.fit(X_train,y_train)

GaussianNB()

# predict the target on the train dataset

predict_train = model.predict(X_train)

predict_train
```

Out:

```
array([1, 1, 1, ..., 0, 0, 0])
```

In:

```
trainaccuracy = accuracy_score(y_train,predict_train)

print('accuracy_score on train dataset : ', trainaccuracy)
```

Out:

```
accuracy_score on train dataset : 0.756603006907761
```

Support Vector Machines:

In:

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(data, y, test_size = 0.20)
```

```
# Applying Support Vector Machine algorithm
```

```
from sklearn.svm import SVC
```

```
svclassifier = SVC(kernel='linear', degree=8)
```

```
svclassifier.fit(X_train, y_train)
```

Out:

```
SVC(C=1.0, cache_size=200, class_weight=None, coef0=0.0,  
    decision_function_shape='ovr', degree=8, gamma='auto_deprecated',  
    kernel='linear', max_iter=-1, probability=False, random_state=None,  
    shrinking=True, tol=0.001, verbose=False)
```

In:

```
y_pred = svclassifier.predict(X_test)
```

```
from sklearn.metrics
```

```
import classification_report, confusion_matrix
```

```
print(confusion_matrix(y_test,y_pred))
```

```
print(classification_report(y_test,y_pred))
```

Out:

```
[[939 109]
```

```
 [183 178]]
```

	precision	recall	f1-score	support
0	0.84	0.90	0.87	1048
1	0.62	0.49	0.55	361
micro avg	0.79	0.79	0.79	1409
macro avg	0.73	0.69	0.71	1409

weighted avg 0.78 0.79 0.78 1409

Linear Discriminant Analysis:

In:

```
control.lda = trainControl(method = "none")
```

```
fit.lda = train(Churn ~ ., data = datatrain, method = "lda", metric = "Accuracy", preProc =  
c("center", "scale"), trControl = control.lda)
```

```
# Test model with 30% test data and obtain accuracy
```

```
predictedval.lda = predict(fit.lda, newdata = datatest, type="raw")
```

```
cm.lda = confusionMatrix(predictedval.lda, datatest$Churn)
```

```
# Results and accuracy
```

```
cm.lda$table
```

```
(cm.lda$overall[1])
```

Out:

Reference

Prediction No Yes

No 1368 237

Yes 180 323

Accuracy: 0.802182163187856

In:

```
predictedprob.lda[predictedprob.lda>=0.6] = "No"
```

```
predictedprob.lda[predictedprob.lda<0.6] = "Yes"
```

```
predictedprob.lda = as.factor(predictedprob.lda)
```

```
cm.ldaprob = confusionMatrix(predictedprob.lda, datatest$Churn)
```

Results and accuracy

cm.ldaprob\$table

(cm.ldaprob\$overall[1])

Out:

Reference

Prediction No Yes

No 1299 181

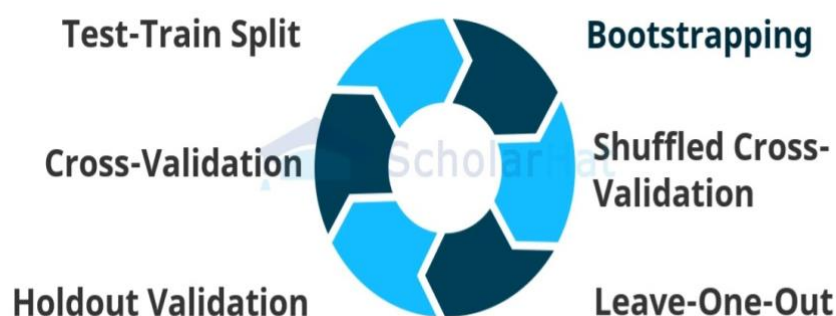
Yes 249 379

MODEL EVALUATION

Model evaluation is the process of using different evaluation metrics to understand a machine learning model's performance, as well as its strengths and weaknesses. Model evaluation is important to assess the efficacy of a model during initial research phases, and it also plays a role in model monitoring.

It is an algorithm trained on a data set to perform a specific predictive task. Model evaluation aims to define how well the model performs its task. The model's performance can vary both across use cases and within a single use case, e.g., by defining different parameters for the algorithm or data selections. Accordingly, we need to evaluate the model's accuracy at each training run. Typically, more than one model is experimented on and productionized for the same use case. This means that we also need to compare multiple models' performance during evaluation.

Model evaluation techniques in machine learning



In:

```
y = dataframe_dummies['Churn'].values  
  
X = dataframe_dummies.drop(columns = ['Churn'])  
  
from sklearn.preprocessing import MinMaxScaler  
  
features = X.columns.values  
  
scaler = MinMaxScaler(feature_range = (0,1))  
  
scaler.fit(X)  
  
X = pand.DataFrame(scaler.transform(X))  
  
X.columns = features
```

In:

```
from sklearn.model_selection import train_test_split  
  
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=101)
```

In:

```
from sklearn.linear_model import LogisticRegression  
  
model = LogisticRegression()  
  
result = model.fit(X_train, y_train)  
  
from sklearn import metrics  
  
from sklearn.metrics import classification_report, confusion_matrix  
  
prediction_test = model.predict(X_test)  
  
# Print results & confusion matrix  
  
print (metrics.accuracy_score(y_test, prediction_test))  
  
print(confusion_matrix(y_test,prediction_test))
```

Out:

0.8075829383886256

Now, to interpret the confusion matrix:

[1418 162]

[244 286]

Let observe from the results that 1418 and 286 were respectively the true positives and false positives, or the correctly classified instances, in this case, the likelihood of a customer to churn. The sum of these instances is 1704, which represents 80% of the total 2110 instances.

MODEL INTERPRETATION

Churn rate, sometimes known as attrition rate, is the rate at which customers stop doing business with a company over a given period of time. Churn may also apply to the number of subscribers who cancel or don't renew a subscription. The higher your churn rate, the more customers stop buying from your business.

Customer Churn Prediction

It is when an existing customer, user, subscriber, or any kind of return client stops doing business or ends the relationship with a company.

Types Of Churn Prediction

- Contractual Churn : When a customer is under a contract for a service and decides to cancel the service e.g. Cable TV, SaaS.
- Voluntary Churn : When a user voluntarily cancels a service e.g. Cellular connection.
- Non-Contractual Churn : When a customer is not under a contract for a service and decides to cancel the service e.g. Consumer Loyalty in retail stores.
- Involuntary Churn : When a churn occurs without any request of the customer e.g. Credit card expiration.

Reason Voluntary Churn

- Lack of usage
- Poor service

- Better price

In:

```
# Churners vs Non-Churners
```

```
dataset['Churn'].value_counts()
```

Out:

```
False    2850
True      483
Name: Churn, dtype: int64
```

In:

```
# Group data by 'Churn' and compute the mean
```

```
print(dataset.groupby('Churn')['Customer service calls'].mean())
```

Out:

```
Churn
False    1.449825
True     2.229814
Name: Customer service calls, dtype: float64
```

In:

```
# Import matplotlib and seaborn
```

```
import matplotlib.pyplot as plt
```

```
import seaborn as sns
```

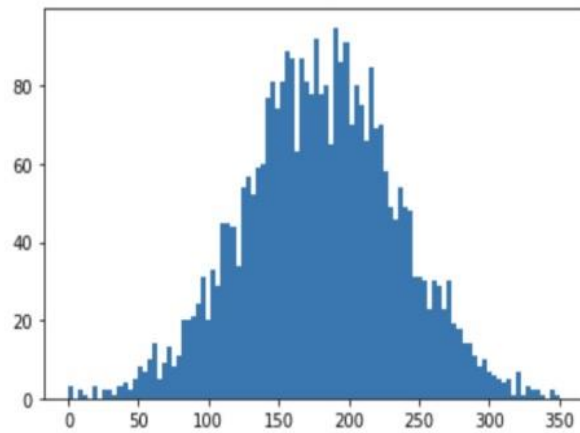
```
# Visualize the distribution of 'Total data minutes'
```

```
plt.hist(dataset['Total day minutes'], bins = 100)
```

```
# Display the plot
```

```
plt.show()
```

Out:



In:

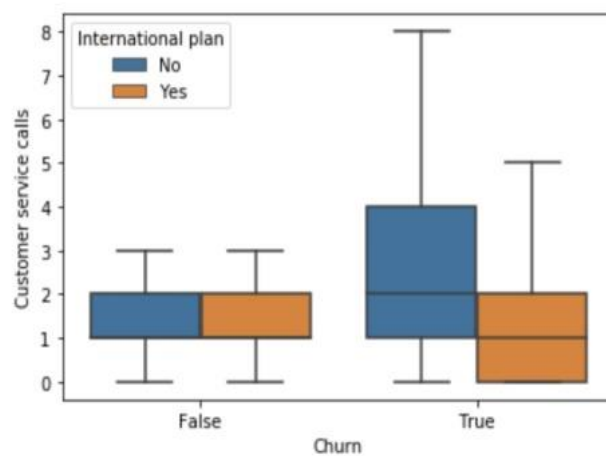
```
# Create the box plot
```

```
sns.boxplot(x = 'Churn', y = 'Customer service calls', data=dataset, sym = "", hue = "International plan")
```

```
# Display the plot
```

```
plt.show()
```

Out:



MODEL DEPLOYMENT

Churn prediction is a critical task for any business that relies on recurring revenue, such as subscription-based services and membership organizations. By identifying customers who are at risk of churning, companies can take steps to retain them and prevent revenue loss. Here going to build and deploy a churn prediction app with Gradio, a platform that allows you to easily create and share interactive web apps with machine learning models.

Gradio

Gradio, much like Streamlit, is “a free and open-source Python library that allows you to develop an easy-to-use customizable component demo for your machine learning model that anyone can use anywhere.” All this can be done with any of a Python script or Jupyter notebook.

Why Deployment?

Deployment is the last stage of the CRISP-DM, the framework used to guide this Customer Churn Prediction project. As stated earlier, churn prediction is a critical task for companies that rely on recurring revenue, and the ability to accurately predict it will be like a superpower for the firms in informing their customer relations, decision making and planning.

Workflow Overview

As indicated earlier, the workflow may be summarized as follows:

- Export ML items
- Set up environment
- Import ML items
- Build interface
- Write the function to process inputs and display outputs
- Deploy the app.

In:

```
# Import Pickle
```

```

import pickle

#Creating a dictionary of objects to export

Exports={"encoder":oh_encoder,

        "scaler":sm_scaler,

        "model":best_xgb_mogel}

#Exporting the dictionary with Pickle0AS FILE

With open("Gradio_app_toolkit","wb")AS FILE:

    Pickle.dump(exports,file)

```

Collect ML Items And Export Them With Pickle

In:

```

# Import OS

import os

# Exporting the requirements

requirements = "\n".join(f"{m.__name__}=={m.__version__}" for m in globals().values() if
getattr(m, "__version__", None))

with open("requirements.txt", "w") as f:

    f.write(requirements)

```

Setting Up Your Environment

In:

```

# Create and activate virtual environment

python -m venv venv; venv\Scripts\activate; python -m pip install -q --upgrade pip; python -m
pip install -qr requirements.txt

```

Importing ML Items

In:

```
# Function to load ML toolkit

def load_ml_toolkit(file_path=r"gradio_src\Gradio_App_toolkit"):

    with open(file_path, "rb") as file:

        loaded_toolkit = pickle.load(file)

    return loaded_toolkit

# Importing the toolkit

loaded_toolkit = load_ml_toolkit(r"gradio_src\Gradio_App_toolkit")

encoder = loaded_toolkit["encoder"]

scaler = loaded_toolkit["scaler"]

# Import the model

model = XGBClassifier()

model.load_model(r"gradio_src\xgb_model.json")

# Exporting the model

best_xgb_model.save_model("xgb_model.json")
```

Building Your Interface

Setting Up The Backend

In:

```
# Output Prediction with Gradio Blocks

output = gr.Label("Awaiting Submission...")

submit_button = gr.Button("Submit")

# Code Block to Process Inputs and Return Outputs
```

```

submit_button.click(fn = process_and_predict,

                    outputs = output,

                    inputs = [gender, SeniorCitizen, Partner, Dependents])

```

For gr.Interface() interface, you may have something like this as a summary of the process:

```

# Output Prediction with gr.Interface()

gr.Interface(inputs=[gender, SeniorCitizen, Partner, Dependents, tenure],

            outputs=gr.DataFrame(headers= ["Churn Status Prediction"]),

            fn=process_and_predict).launch(inbrowser= True)

```

In:

```

# Launch the app in Gradio Blocks

gr_app.launch(inbrowser= True, share=True)

```

The “inbrowser = True” argument is specified so that the app loads in browser when the script is run.

EXPLORATORY DATA ANALYSIS

Demographic Info About Customers

1- Gender

In:

```

gndr_crn=df.groupby(['gender','Churn']).apply(lambda
x:x['Churn'].count()).reset_index(name='Counts')

```

In:

```

linkcode

```

```

gndr_crn

```

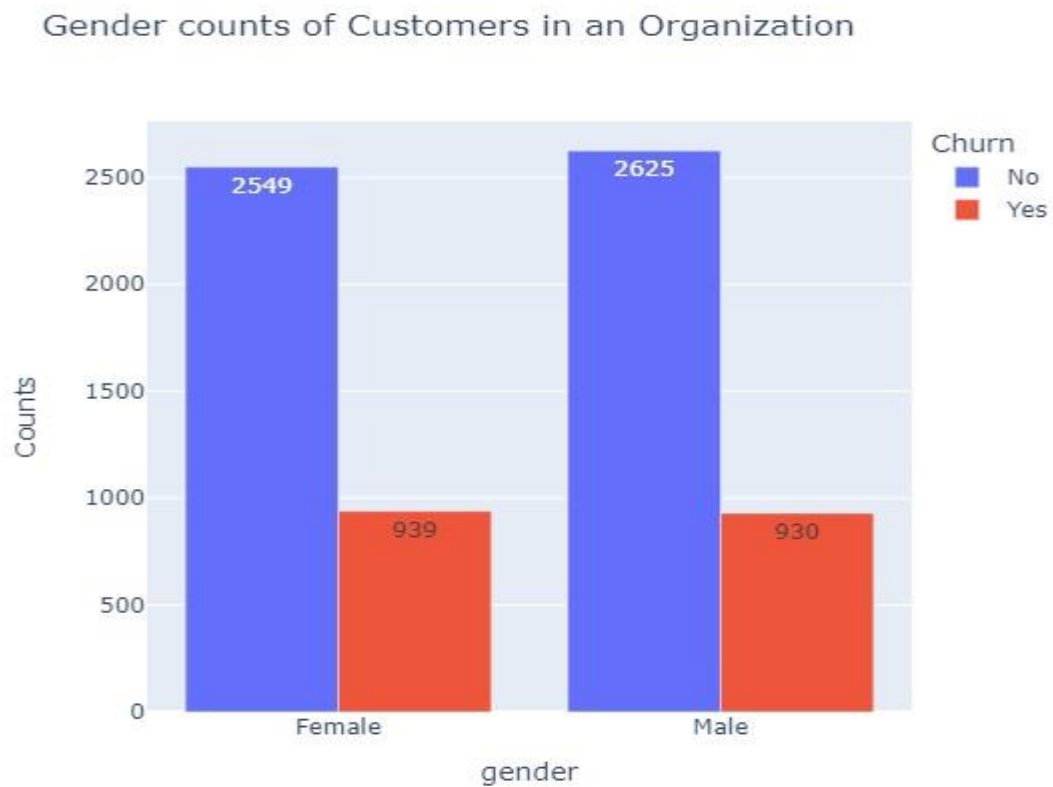
Out:

	gender	Churn	Counts
0	Female	No	2549
1	Female	Yes	939
2	Male	No	2625
3	Male	Yes	930

In:

```
fig=px.bar(gndr_crn,x='gender',y='Counts',color='Churn',text_auto=True,  
           title='Gender counts of Customers in an Organization')  
  
fig.update_layout(barmode='group', xaxis_tickangle=0)  
  
fig.show()
```

Out:



2- age

In:

```
snrctzn_crn=df.groupby(['SeniorCitizen','Churn']).apply(lambdax:x['Churn'].count()).reset_in  
dex(name='Counts')
```

```
fig=px.bar(snrctzn_crn,x='SeniorCitizen',y='Counts',color='Churn',  
text_auto=True,title='Gender counts of Customers in an Organization')
```

```
fig.update_layout(barmode='group', xaxis_tickangle=0)
```

```
fig.show()
```

Out:



In:

```
per=np.array([(4508/(4508+1393)),(1393/(4508+1393)),(666/(666+476)),(476/(666+476))]).  
round(2)
```

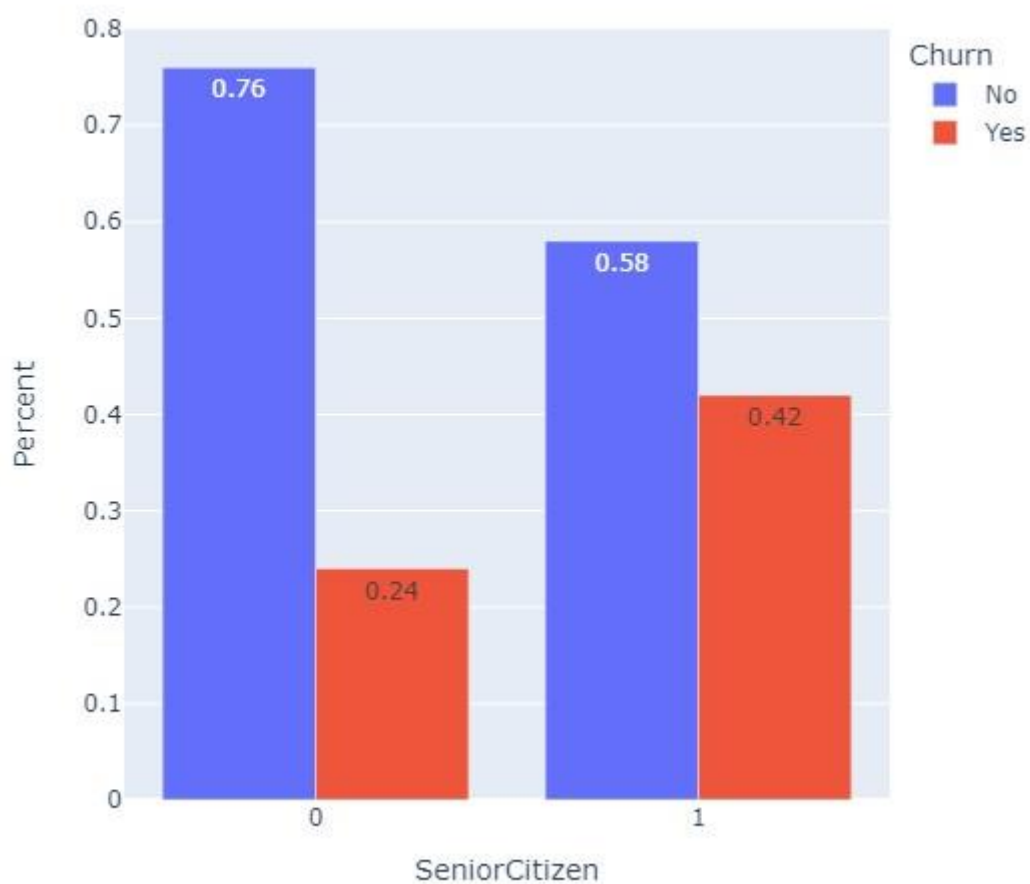
```
snrctzn_crn['Percent']=per
```

```
fig=px.bar(snrctzn_crn,x='SeniorCitizen',y='Percent',color='Churn', text_auto=True)
```

```
fig.update_layout(barmode='group', xaxis_tickangle=0)
```

```
fig.show()
```

Out:

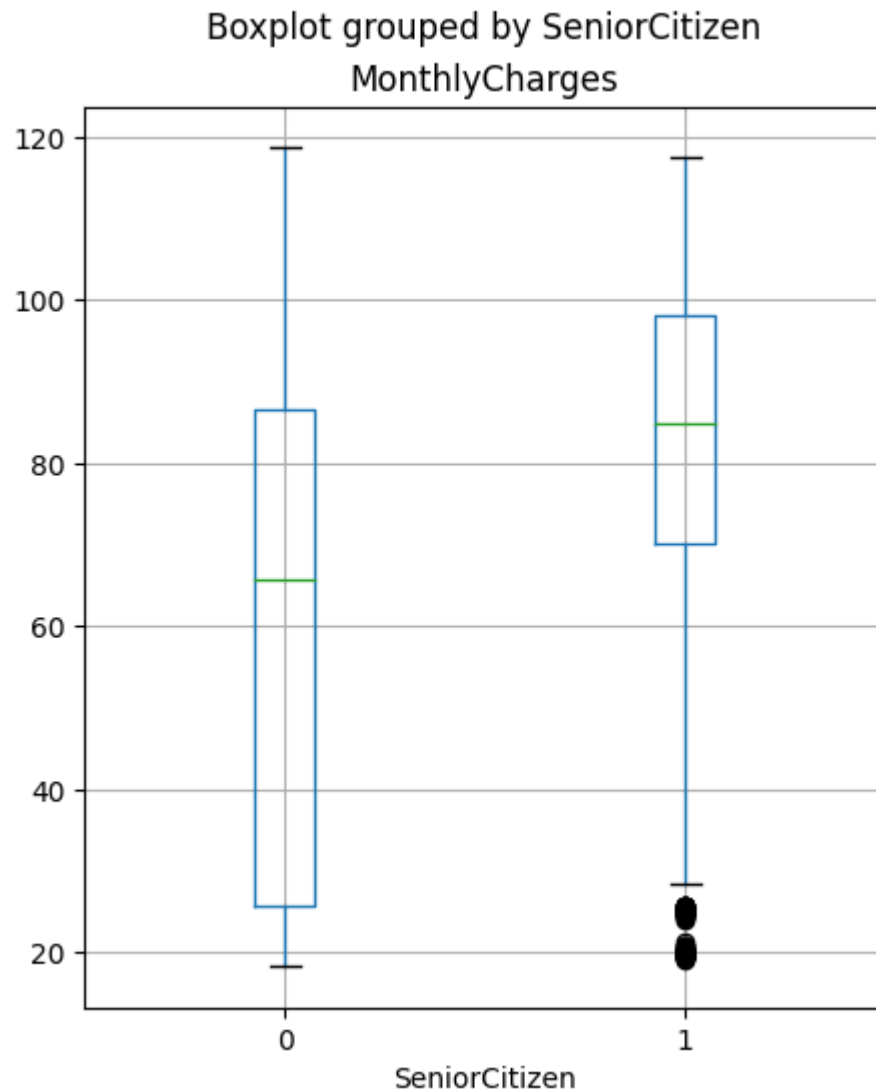


Although the percentage of elderly people is small compared to the number of customers, the percentage of elderly people who have dispensed with the service is very large, reaching 42%.

```
df.boxplot(column='MonthlyCharges', by='SeniorCitizen', figsize=(5,6))
```

Out:

```
<Axes: title={'center': 'MonthlyCharges'}, xlabel='SeniorCitizen'>
```



3- partner

In:

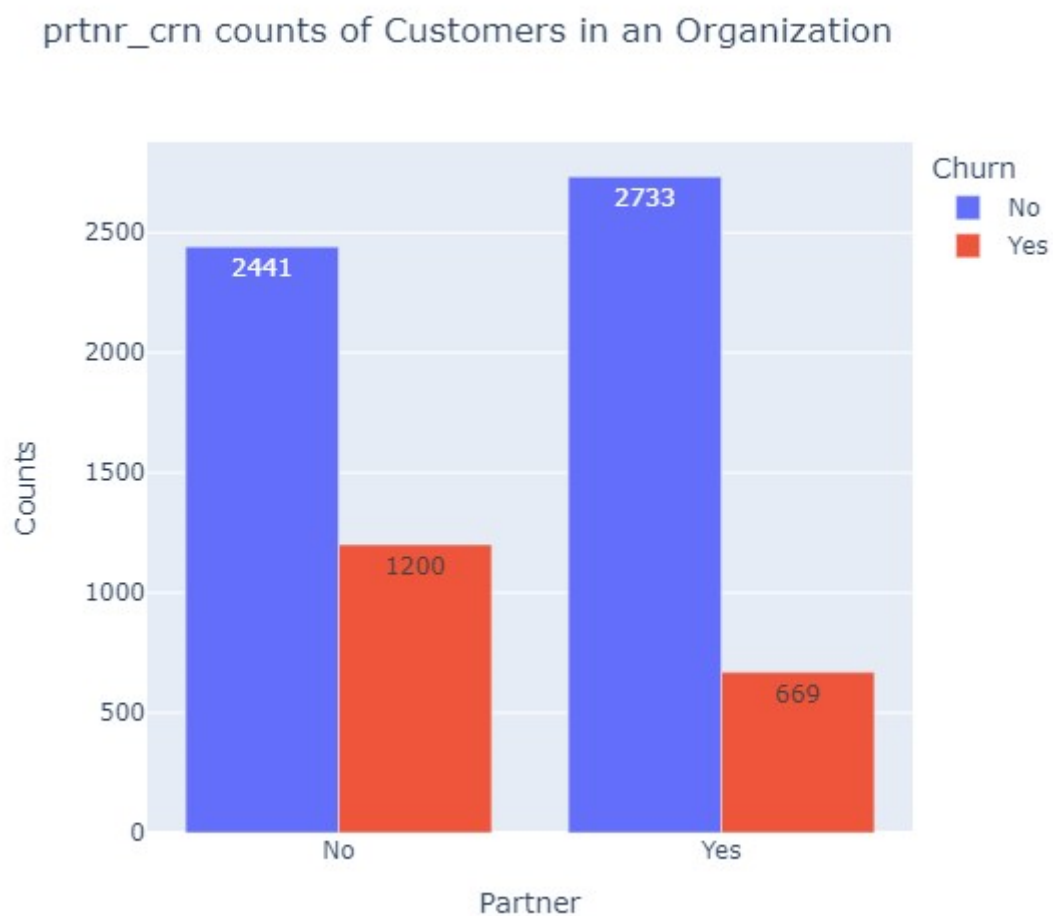
[illegible]

```
fig=px.bar(prtnr_crn,x='Partner',y='Counts',color='Churn',text_auto=True,title='prtnr_crn
counts of Customers in an Organization')
```

```
fig.update_layout(barmode='group', xaxis_tickangle=0)
```

```
fig.show()
```

Out:



In:

```
per=np.array([(2441/(2441+1200)),(1200/(2441+1200)),(2733/(669+2733)),(669/(669+2733))
)).round(2)
```

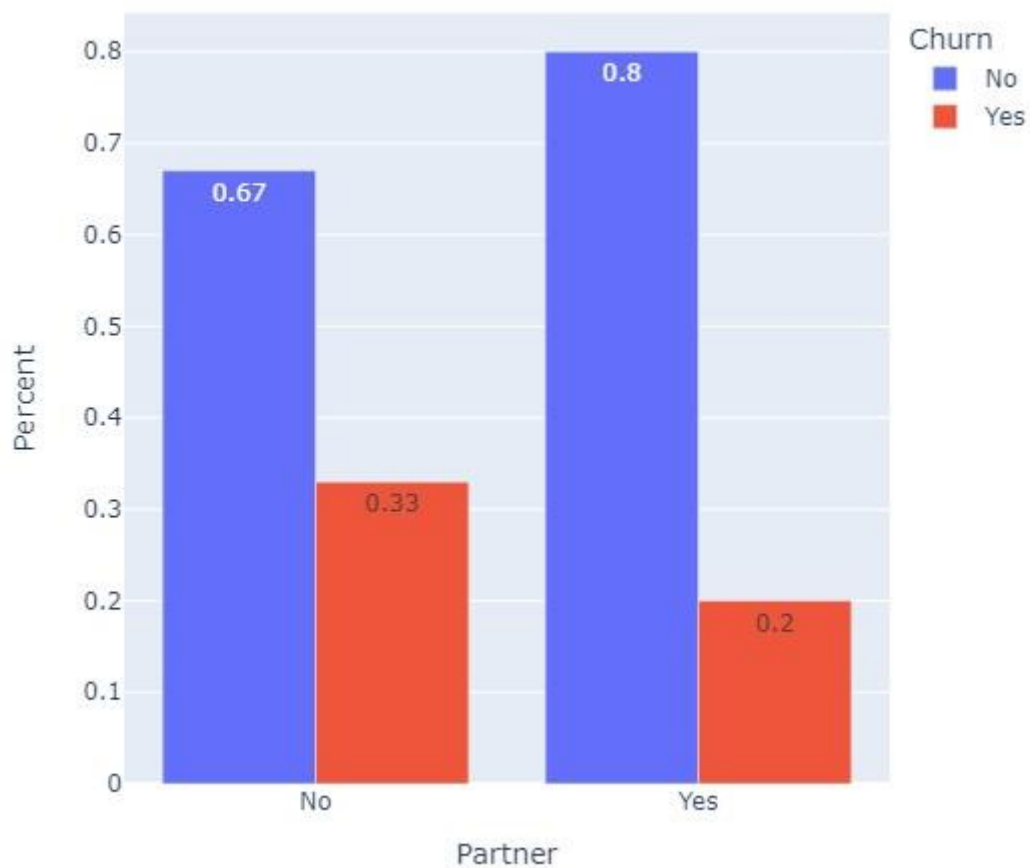
```
prtnr_crn['Percent']=per
```

```
fig=px.bar(prtnr_crn,x='Partner',y='Percent',color='Churn', text_auto=True)
```

```
fig.update_layout(barmode='group', xaxis_tickangle=0)
```

```
fig.show()
```

Out:



Customers who leave the service and have no partners have higher churn rate than who leave the service and have partners.

Services that each customer has signed up for

- phone, multiple lines, internet, online security, online backup, device protection, tech support, and streaming TV and movies

In:

```
col=['PhoneService','MultipleLines','InternetService','OnlineSecurity',
     'OnlineBackup','DeviceProtection','TechSupport','StreamingTV','StreamingMovies']

fig,axs=plt.subplots(3,3,figsize=(20,20))

axs=axs.flatten()

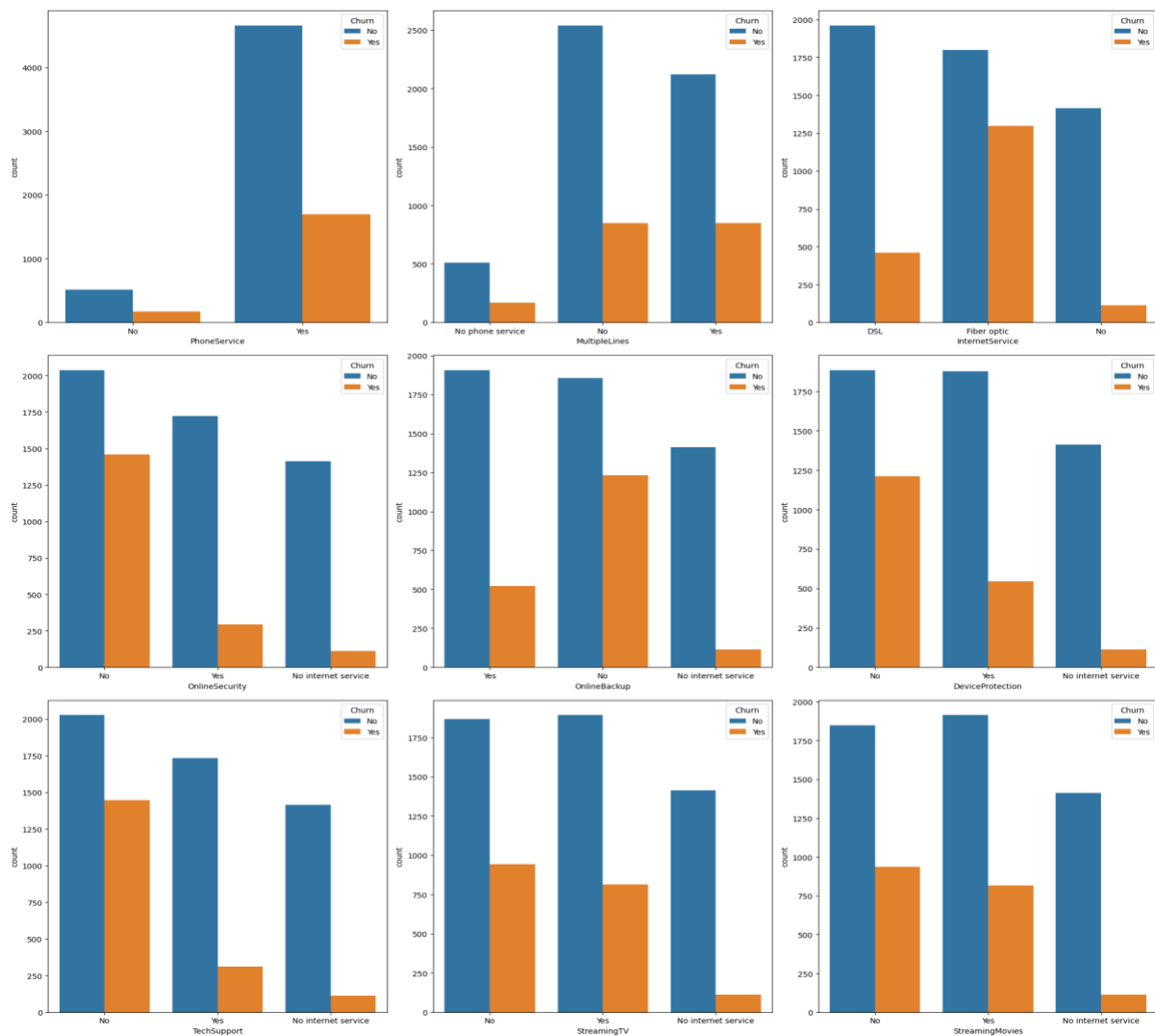
for i,var in enumerate(col):

    sns.countplot(data=df,x=var,hue='Churn',ax=axs[i])

plt.tight_layout()

plt.show()
```

Out:



A high number of customers have displayed their resistance towards the use of Fiber optic cables for providing the InternetService. On the contrary, from the above graph, customers prefer using DSL for their InternetService.

In:

```
cols=col=['PhoneService','MultipleLines','InternetService','OnlineSecurity',  
          'OnlineBackup','DeviceProtection','TechSupport','StreamingTV','StreamingMovies']
```

for name in cols:

```
    n=df[name].nunique()  
  
    col=df[name].value_counts(normalize=True)  
  
    # Creating plot  
  
    explode = (0.0,)*n  
  
    # Creating color parameters  
  
    colors = ( "green", "red","blue",'orange')  
  
    # Wedge properties  
  
    wp = { 'linewidth' : 1, 'edgecolor' : "white" }  
  
    # Creating autocpt arguments  
  
    def func(pct, allvalues):  
  
        absolute = int(pct / 100.*np.sum(allvalues))  
  
        return "{:.1f}%".format(pct, absolute)
```

Creating plot

```
fig, ax = plt.subplots(figsize =(7, 3))  
  
wedges, texts, autotexts = ax.pie(col,  
  
                                   autopct = lambda pct: func(pct, col),  
  
                                   explode = explode,  
  
                                   labels = col.index,
```

```

        shadow = True,

        colors = colors,

        startangle = 90,

        wedgeprops = wp,

        textprops = dict(color ="black"))

# Adding legend

ax.legend(wedges, att.index,

        title ="Churn",

        loc ="center left",

        bbox_to_anchor =(1, 0, 0.5, 1))

plt.setp(autotexts, size = 8, weight ="bold")

ax.set_title(f"Propotion Of {name}")

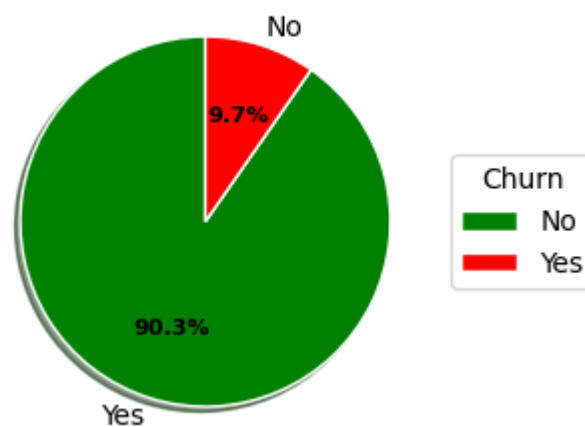
# show plot

plt.show()

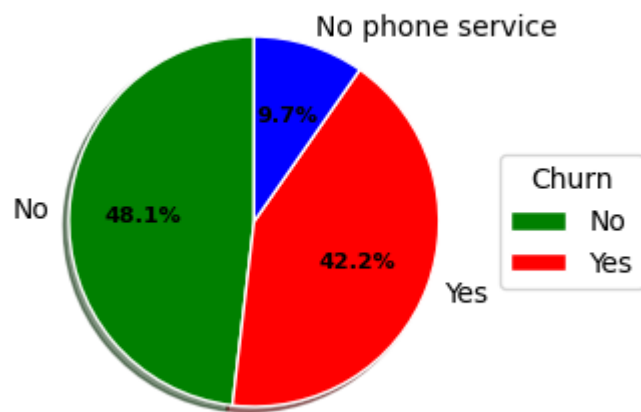
```

Out:

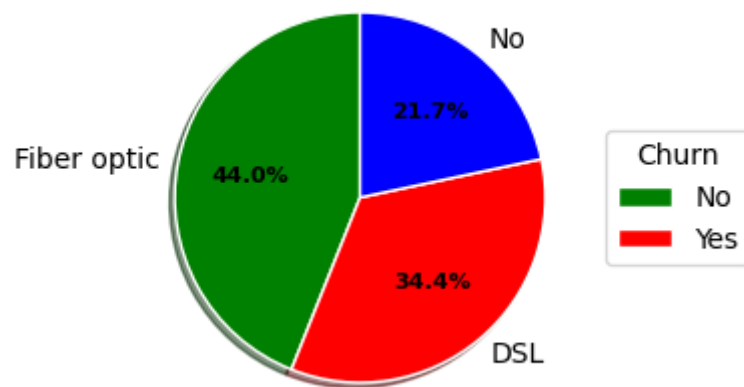
Propotion Of PhoneService



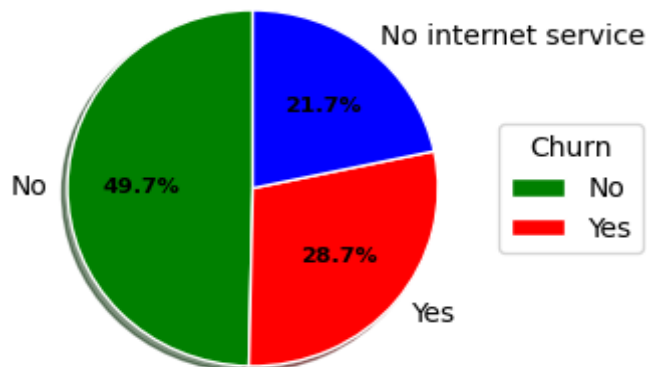
Propotion Of MultipleLines



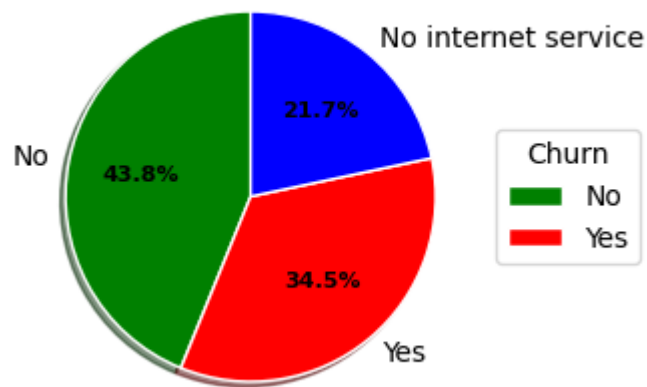
Propotion Of InternetService



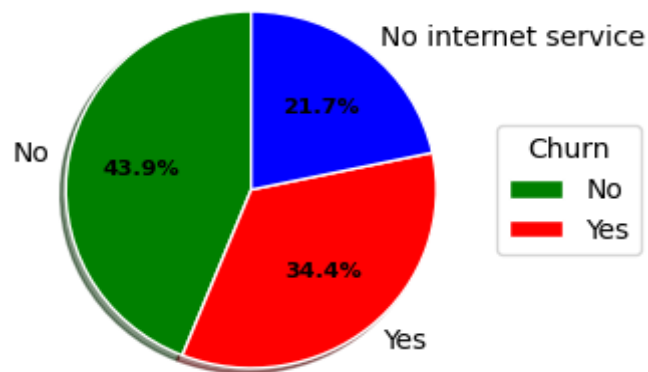
Propotion Of OnlineSecurity



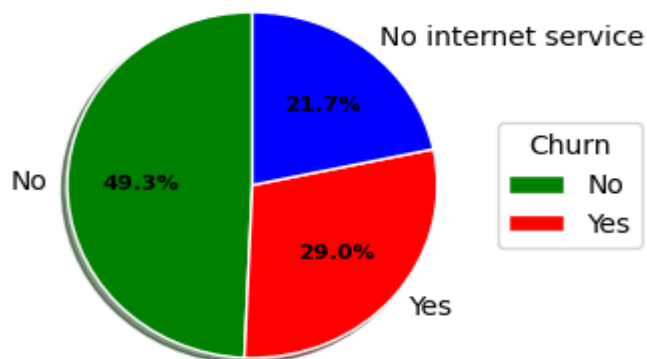
Propotion Of OnlineBackup



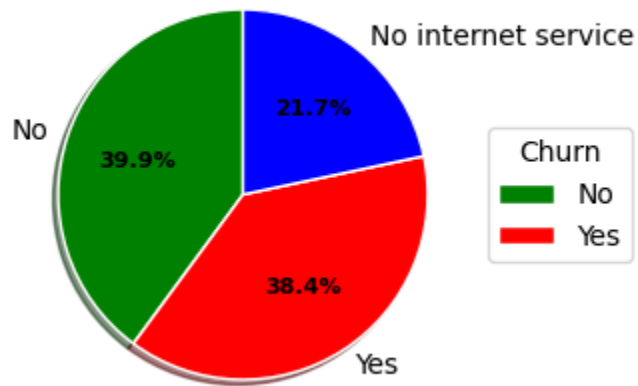
Propotion Of DeviceProtection



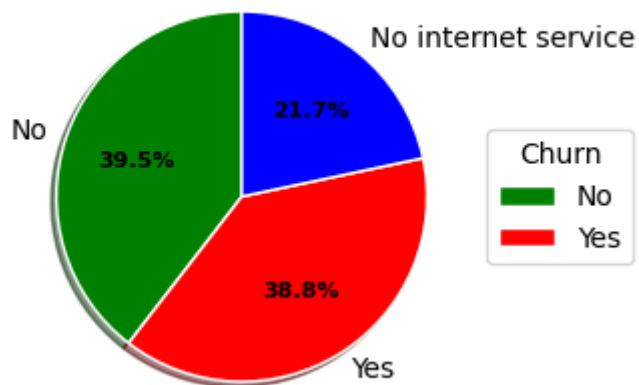
Propotion Of TechSupport



Propotion Of StreamingTV



Propotion Of StreamingMovies



Services Subscribed By The Customer :

In:

```
col=['Contract','PaperlessBilling','PaymentMethod']
```

```
fig,axs=plt.subplots(1,3,figsize=(15,5))
```

```
axs=axs.flatten()
```

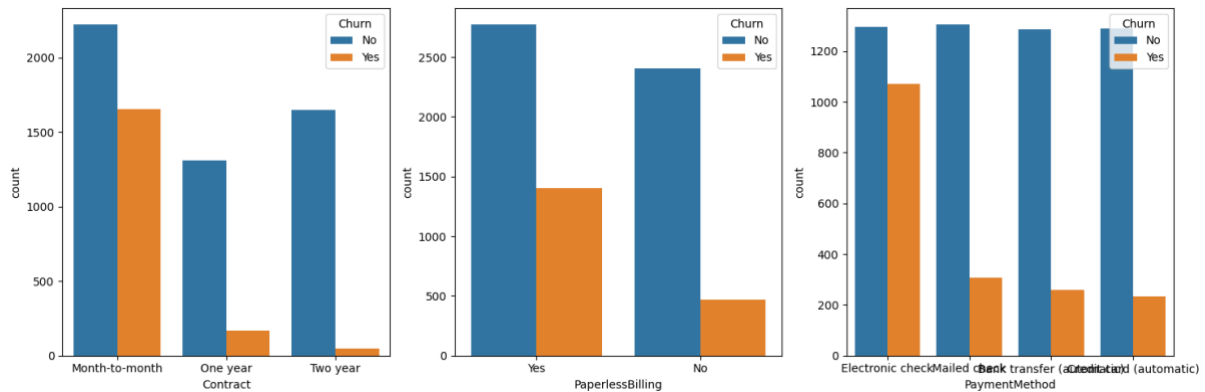
```
for i,var in enumerate(col):
```

```
sns.countplot(data=df,x=var,hue='Churn',ax=axes[i])
```

```
plt.tight_layout()
```

```
plt.show()
```

Out:



- When it comes down to catering the customers, services w.r.t OnlineSecurity, OnlineBackup, DeviceProtection & TechSupport are crucial from the above visualizations!
- A high number of customers have switched their service provider when it comes down poor services with the above mentioned features.

In:

```
cols=['Contract','PaperlessBilling','PaymentMethod']
```

```
for name in cols:
```

```
    n=df[name].nunique()
```

```
    col=df[name].value_counts(normalize=True)
```

```
    # Creating plot
```

```
    explode = (0.0,)*n
```

```
# Creating color parameters
```

```
    colors = ( "green", "red","blue",'orange')
```

```
    # Wedge properties
```

```

wp = { 'linewidth' : 1, 'edgecolor' : "white" }

# Creating autocpt arguments

def func(pct, allvalues):

    absolute = int(pct / 100.*np.sum(allvalues))

    return "{:.1f}%".format(pct, absolute)

# Creating plot

fig, ax = plt.subplots(figsize=(7, 3))

wedges, texts, autotexts = ax.pie(col, autopct = lambda pct: func(pct, col),

                                explode = explode,

                                labels = col.index,

                                shadow = True,

                                colors = colors,

                                startangle = 90,

                                wedgeprops = wp,

                                textprops = dict(color = "black"))

# Adding legend

ax.legend(wedges, att.index,

          title = "Churn",

          loc = "center left",

          bbox_to_anchor=(1, 0, 0.5, 1))

plt.setp(autotexts, size = 8, weight = "bold")

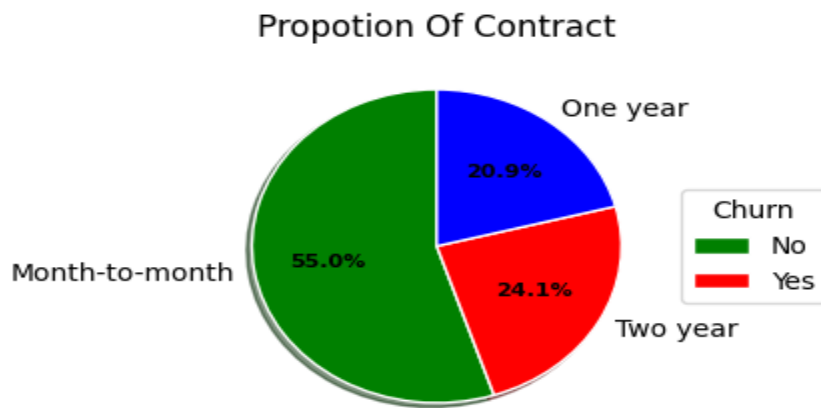
ax.set_title(f'Propotion Of {name}')

# show plot

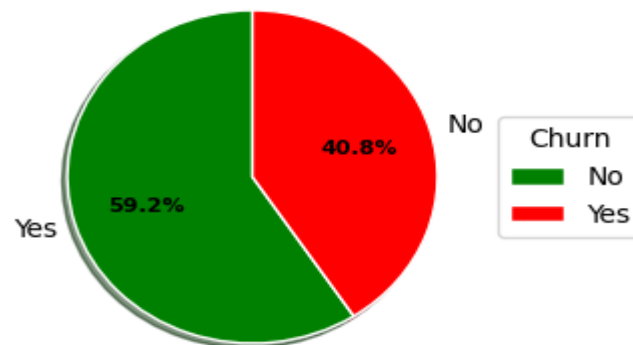
```

```
plt.show()
```

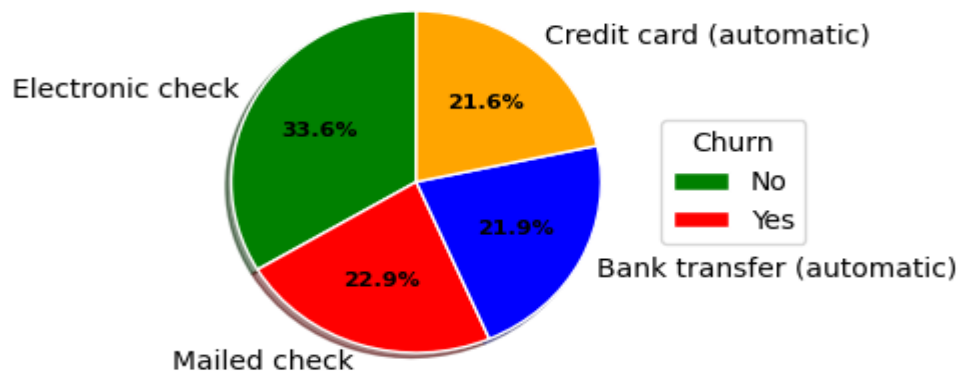
Out:



Propotion Of PaperlessBilling



Propotion Of PaymentMethod



BUILD MODEL

Split

In:

```
target = "Churn"

X = df.drop(columns=[target,'customerID'])

y = df[target]

print("X shape:", X.shape)

print("y shape:", y.shape)
```

Out:

```
X shape: (7043, 19)

y shape: (7043,)
```

In:

```
X_train, X_test, y_train, y_test = train_test_split(X,y,test_size=0.2,random_state=42)

print("X_train shape:", X_train.shape)

print("y_train shape:", y_train.shape)

print("X_test shape:", X_test.shape)

print("y_test shape:", y_test.shape)
```

Out:

```
X_train shape: (5634, 19)

y_train shape: (5634,)

X_test shape: (1409, 19)

y_test shape: (1409,)
```

1- RandomForestClassifier

In:

linkcode

```
# Fit on `X_train`, `y_train`
```

```
Dec_model = make_pipeline(OneHotEncoder(use_cat_names=True),StandardScaler(),  
                           RandomForestClassifier(criterion='gini',
```

```
max_depth=50,
```

```
min_samples_split=2,
```

```
min_samples_leaf=1,
```

```
min_weight_fraction_leaf=0.0,
```

```
max_features=None,
```

```
random_state=None,
```

```
max_leaf_nodes=None,
```

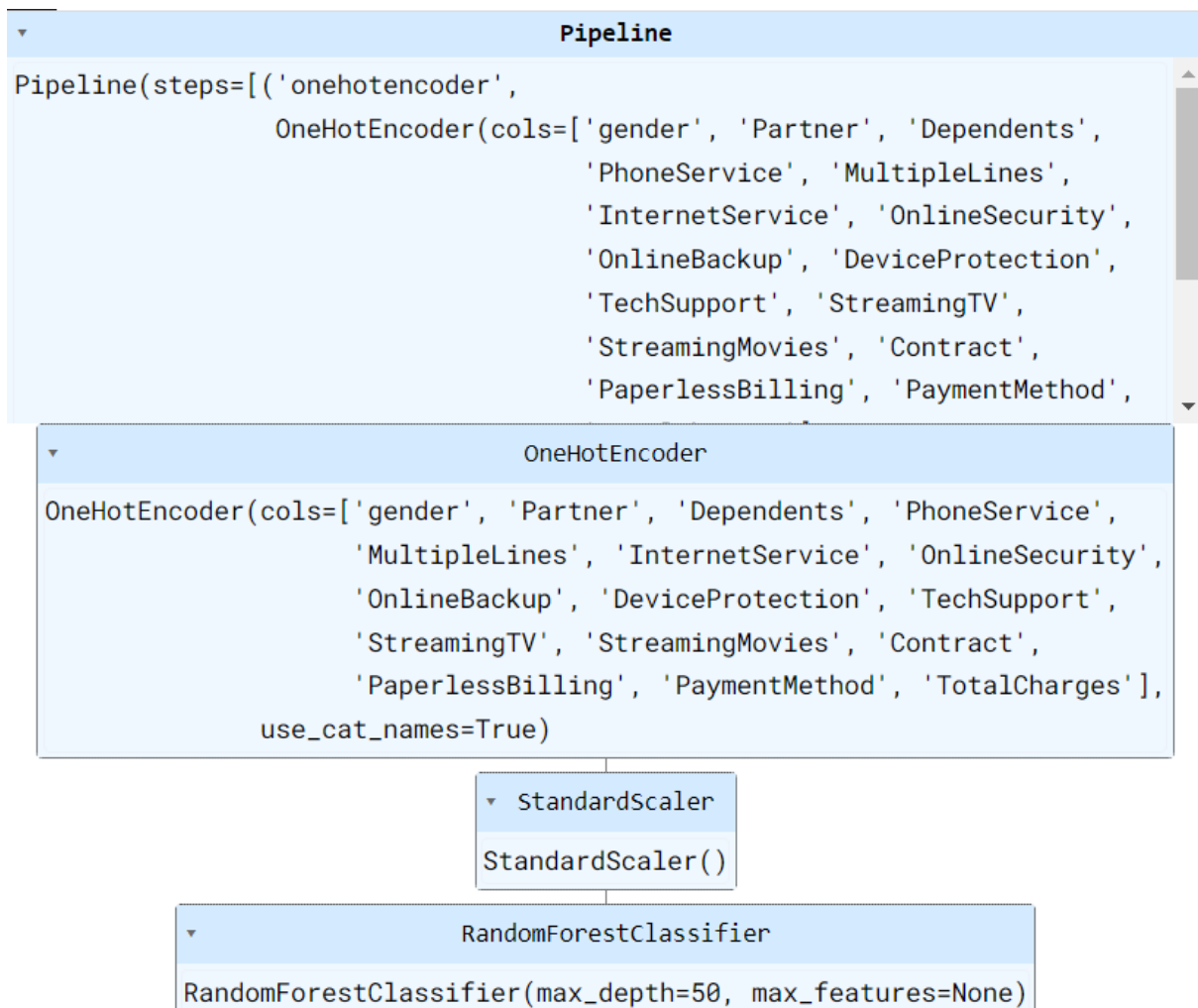
```
min_impurity_decrease=0.0,
```

```
class_weight=None,
```

```
ccp_alpha=0.0,))
```

```
Dec_model.fit(X_train, y_train)
```

Out:



In:

```
acc_train = Dec_model.score(X_train,y_train)  
  
acc_test = Dec_model.score(X_test,y_test)  
  
print("Training Accuracy:", round(acc_train, 4))  
  
print("Test Accuracy:", round(acc_test, 4))
```

Out:

Training Accuracy: 0.9977

Test Accuracy: 0.7984

In:

```
# Print classification report
```

```
print(classification_report(y_test,Dec_model.predict(X_test)))
```

Out:

```
precision  recall  f1-score  support

No         0.83     0.91     0.87     1036
Yes        0.66     0.50     0.57      373

accuracy               0.80     1409
macro avg      0.75     0.70     0.72     1409
weighted avg   0.79     0.80     0.79     1409
```

CONCLUSION

In conclusion, our customer churn prediction project has successfully culminated in the creation of a powerful tool that holds the potential to transform how businesses manage customer retention. Throughout this journey, we have analyzed, preprocessed, and engineered data, explored various machine learning algorithms, and rigorously tested our predictive model. The results are promising, as our model demonstrates its ability to identify customers at risk of churning with a high degree of accuracy.

By implementing this solution, businesses can proactively address churn, thereby improving customer retention rates and enhancing long-term profitability. It is important to emphasize that this project represents just the beginning of a continuous improvement cycle. To maximize its impact, ongoing data collection, model refinement, and real-time implementation are essential.

In an era where customer loyalty and satisfaction are paramount, our churn prediction model equips businesses with the insights needed to cultivate enduring customer relationships and secure a competitive edge in the market. As we look ahead, the journey continues, as we refine and enhance our solution to meet the ever-evolving needs of the business landscape. Customer churn prediction is not just a project; it's a strategic advantage for those committed to staying at the forefront of customer-centric practices and ensuring lasting success in their respective industries.